

SAZERAC COMPANY

VMI & Inventory Planning Team | Version 2

Table of Contents

1	Overview	3
2	Primary View: vw_Orders_Base	3
3	Column Reference — vw_Orders_Base	4
4	Related Views	5
5	View Comparison Matrix	6
6	Hold Code Logic	7
7	Recommended Query Patterns	7
8	Data Layer Recommendation	8
9	Known Risks & Open Items	8

1. Overview

This document provides technical reference for the SQL views used by the Sazerac VMI & Inventory Planning team when querying Open State (Oracle ERP) sales order data. It documents the primary view — **vw_Orders_Base** — explains the design decisions behind it, and describes the other order-related views and when each should (or should not) be used.

All views sit within the **dbo** schema and draw from Oracle SCM Distributed Order Orchestration (DOO) extract tables. They cover **Open State** orders only. System21 / Bailment data is held separately in **SYS21_OpenOrders** and is outside scope.

■ Charge lines (e.g., PALLETSLIPCHG) appear in this view alongside product lines. Always filter **WHERE SKU != 'PALLETSLIPCHG'** (or equivalent) for any inventory or quantity-based analysis. Charge lines may also return NULL for Company, Plant, Stockroom, and OrganizationCode.

2. Primary View: vw_Orders_Base

2.1 What It Is

vw_Orders_Base is the foundational orders view for Open State replenishment analysis. It returns one row per order line covering all fields needed for order status tracking, quantity reconciliation, customer identification, and downstream joins to inventory and planning data. It includes both open and shipped lines, making it suitable for current-state queries and historical analysis.

2.2 Why vw_Orders_Base Is the Correct Foundation

- **Fulfill-line deduplication is handled correctly.** A `ROW_NUMBER()` CTE (*LatestFulfillLine*) retains only the most recent fulfill line record per order line. Without this, rescheduled or amended lines in Oracle DOO produce duplicate rows and overstate quantities.
- **Draft and reference orders are excluded at source.** Header join filters out `DOO_DRAFT` and `DOO_REFERENCE` status codes and rows where `HEADERSOURCEDOCUMENTTYPECODE IS NOT NULL`.
- **Full order lifecycle is preserved.** No pre-filter on ship date or status. Analysts apply their own filters (e.g., `WHERE ShipDate IS NULL` for open lines) rather than being constrained by logic baked into the view.
- **OrdersBaseKey enables clean downstream joins.** A composite key (SKU + Account + Company + DeliveryAddressCode + OrganizationCode) is pre-computed for joining to inventory and planning datasets.
- **Shipped quantity is available.** `FulfillLineShippedQTY` allows direct ordered-vs-shipped comparison for OTIF analysis and short-ship identification.

2.3 Underlying Tables

Table	Role
OracleSCMDistributedOrderOrchestrationLine Extract	Order line — qty, SKU, status, dates
OracleSCMDistributedOrderOrchestrationHeaderExtract	Order header — customer, type, hold flag

Table	Role
OracleSCMDistributedOrderOrchestrationFulfillLineExtract	Fulfill line — shipped qty, sub-inv, org (deduplicated via CTE)
OracleSCMDistributedOrderOrchestrationOrderAddressExtract	Ship-to address linkage
OracleCommonPartySite	Party site — delivery address code extraction
OracleCommonCustomerAccount	Customer account number
OracleCommonPartyExtract	Customer name
OracleCommonItemExtract	SKU number and description
OracleSCMCommonInvOrgParametersExtract	Org / plant attributes (Company, Plant codes)
OracleCommonOrganizationUnit	Organization name
OracleCustomSalesOrdersFullFillLineEFF	Extended flex fields — allocation details context
OracleCustomSubInventoryDetails	Sub-inventory / stockroom mapping

3. Column Reference — vw_Orders_Base

■ **CustomerNumber and AccountNumber are redundant — both return the account number (e.g., '1057087'). Similarly, CustomerName and AccountName return identical values. Use CustomerNumber and CustomerName for consistency; treat the Account equivalents as legacy duplicates.**

Column	Type	Nullabl e	Description / Validation Notes
PONumber	varchar	Yes	Customer PO number (e.g., '80729036-9')
OrderNumber	varchar	Yes	Oracle DOO order number (e.g., '5210798')
LineNumber	varchar	Yes	Display line number; SYS21-sourced lines use source line number
OrderDate	datetime2	Yes	Date/time the order was placed in Oracle
HeaderCreateDate	date	Yes	Date the order header was created
HeaderOnHold	varchar	Yes	Header-level hold flag — Y or N
LineOnHold	varchar	Yes	Line-level hold flag — Y or N
LineCreateDate	date	Yes	Date the order line was created
RequestedShipDate	date	Yes	Customer-requested ship date (RSD)
ShipDate	date	Yes	Actual ship date — NULL if not yet shipped (open lines)
OrderStatus	varchar	Yes	Header status (observed: OPEN; also CLOSED, SHIPPED)
LineStatusCode	varchar	Yes	Line status (observed: OPEN)
HeaderTypeCode	varchar	Yes	Order type (observed: STANDARD ORDER)
OrderQty	float	Yes	Ordered quantity in LineUOM. Exclude charge lines for product analysis.
FulfillLineShippedQTY	float	Yes	Quantity confirmed shipped. NULL until shipment is confirmed.
SKUID	bigint	Yes	Internal Oracle inventory item ID (e.g., 100000300904388)
SKU	varchar	Yes	Sazerac SKU number. Note: 'PALLETSLIPCHG' is a charge line, not a product.
SKUDesc	varchar	Yes	SKU description (e.g., 'WHEATLEY VODKA 6/1.75L 82PF')
LineUOM	varchar	Yes	Unit of measure — CS (case) for product lines, EA for charges
Company	varchar	Yes	2-char company code (e.g., SA, SI). NULL on charge lines.
Plant	varchar	Yes	2-char plant code (e.g., 76, 45). NULL on charge lines.
Stockroom	varchar	Yes	2-char stockroom code. NULL in observed data — populates under specific allocation conditions.
OrganizationCode	varchar	Yes	Oracle org code (e.g., '032', '033'). NULL on charge lines.
OrganizationName	varchar	Yes	Full org name (e.g., 'Jeffersonville Distribution [SA76]')
LineSubInventory	varchar	Yes	Sub-inventory from fulfill line. NULL in observed data — likely populates post-shipment or for specific orgs.

Column	Type	Nullable	Description / Validation Notes
CustomerID	bigint	Yes	Internal Oracle sold-to party ID. Safe join key to Oracle party tables.
CustomerNumber	varchar	Yes	Customer account number (e.g., '1057087'). Same value as AccountNumber.
DeliveryAddressCode	varchar	Yes	Delivery location code extracted from party site number (e.g., '001', '027')
CustomerName	varchar	Yes	Customer/distributor name (e.g., 'PLCB', 'GOLDEN BRANDS - REYES HOLDINGS'). Same as AccountName.
AccountNumber	varchar	Yes	Redundant with CustomerNumber. Same value returned.
AccountName	varchar	Yes	Redundant with CustomerName. Same value returned.
OrdersBaseKey	varchar	No	Composite join key: SKU + AccountNumber + Company + DeliveryAddressCode + OrganizationCode. Only NOT NULL column — safe join anchor.

4. Related Views

4.1 vw_Orders

vw_Orders is a parallel view — independently built against the same Oracle extract tables as **vw_Orders_Base**. It adds City/State geography, aggregated hold codes, and parsed pricing strategy fields. However, it has a critical structural deficiency: the fulfill line is joined with a raw `LEFT JOIN` with no deduplication. When an order line has more than one fulfill line record (common after reschedules or amendments), **vw_Orders** returns **duplicate rows**, making quantity aggregations unreliable.

`OrderQty` and `LineUnitSellingPrice` are prefixed with underscores (`_OrderQty`, `_LineUnitSellingPrice`) — these columns appear to be flagged or in transition. Confirm intent with jsimpson@sazerac.com before relying on them.

Note: Appropriate for display-only queries where hold code or pricing visibility is needed. Not safe for quantity aggregation or order counting without explicit deduplication.

4.2 vw_OpenOrdersWithHold

Filters to open orders with at least one active hold. Schema identical to `vw_Orders`; inherits the same fulfill-line duplication risk. Useful for hold-monitoring dashboards or exception queues.

4.3 vw_OpenOrdersWithNPHold

Identical schema to `vw_OpenOrdersWithHold`; filters specifically to **Non-Pricing (NP) holds**. NP holds indicate fulfillment or logistics issues — the highest-priority exception type for the planning team. Use this view when triaging NP-hold exceptions.

4.4 vw_OrderFulfillmeantLineHoldCodes

Lightweight lookup: `LineId` + `HoldCodes` (comma-separated). Can be joined directly to **vw_Orders_Base** to add hold visibility without adopting `vw_Orders`' duplication risk. This is the recommended pattern for agent and reporting queries requiring hold codes alongside clean order data.

4.5 SYS21_OpenOrders (Bailment / Control State)

Separate base table from System21 for Bailment and Control State markets. Entirely outside the Oracle DOO pipeline. Out of scope for Open State analysis; do not join to Oracle order views without explicit reconciliation logic.

5. View Comparison Matrix

Attribute	vw_Orders_Base	vw_Orders	vw_OpenOrdersWithHold	vw_OpenOrdersWithNPHold
Fulfill line deduplicated	YES	NO	NO	NO
Draft/reference orders excluded	YES	YES	YES	YES
Shipped lines included	YES	YES	NO	NO
ShipDate / ShippedQTY	YES	NO	NO	NO
PONumber	YES	NO	NO	NO
CustomerID (internal key)	YES	NO	NO	NO
OrdersBaseKey join column	YES	NO	NO	NO
City / State	NO	YES	YES	YES
Hold codes (aggregated)	NO*	YES	YES	YES
Pricing strategy fields	NO	YES	YES	YES
Safe for qty aggregation	YES	RISK	RISK	RISK
Pre-filtered to held orders	NO	NO	All	NP only
Charge lines present	YES	YES	YES	YES

Note: * Hold codes not in vw_Orders_Base natively. Join vw_OrderFulfillmentLineHoldCodes on LineId to add hold visibility while preserving deduplication integrity.

6. Hold Code Logic

Hold codes are managed at two entity levels in Oracle DOO and must be captured from both for a complete picture:

Entity Level	Oracle Entity Name	What It Covers
Fulfill Line	DOO_ORDER_FLINE_V	Holds at fulfillment line level — tied directly to a specific ship action
Order Line	DOO_ORDER_LINES_V	Holds at order line level — covers the line before fulfillment is created

The **lineholds CTE** in **vw_Orders** captures both levels via a UNION ALL and aggregates them with STRING_AGG into a comma-separated string per line. This CTE can be extracted and applied as a standalone join against vw_Orders_Base without inheriting vw_Orders' duplication problem.

NP Holds (Non-Pricing) are the highest-priority exception type for the planning team — they indicate fulfillment/logistics issues rather than commercial pricing disputes. **vw_OpenOrdersWithNPHold** is the dedicated exception queue view.

7. Recommended Query Patterns

7.1 Open Orders for a Customer (Product Lines Only)

```
SELECT OrderNumber, LineNumber, SKU, SKUDesc, OrderQty,
FulfillLineShippedQTY, RequestedShipDate, OrderStatus
FROM vw_Orders_Base
WHERE CustomerNumber = '<customer_number>'
AND ShipDate IS NULL
AND SKU != 'PALLETSLIPCHG'
ORDER BY RequestedShipDate;
```

7.2 Open Orders With Hold Codes

```
WITH lineholds AS (
-- Port the lineholds CTE from vw_Orders exactly as written
-- (captures holds at D00_ORDER_FLINES_V and D00_ORDER_LINES_V)
)
SELECT ob.*, lh.HoldCodes
FROM vw_Orders_Base ob
LEFT JOIN lineholds lh ON <LineId join key> -- confirm key before use
WHERE ob.ShipDate IS NULL
AND ob.SKU != 'PALLETSLIPCHG'
AND lh.HoldCodes IS NOT NULL;
```

Note: vw_Orders_Base does not expose LineId directly. Confirm join key against OracleSCMDistributedOrderOrchestrationLineExtract.LINEID before finalizing.

7.3 Ordered vs. Shipped Quantity Reconciliation

```
SELECT SKU, SKUDesc, CustomerName,
SUM(OrderQty) AS TotalOrdered,
SUM(FulfillLineShippedQTY) AS TotalShipped,
SUM(OrderQty) - SUM(FulfillLineShippedQTY) AS OpenBalance
FROM vw_Orders_Base
WHERE Company = 'SA'
AND SKU != 'PALLETSLIPCHG'
AND RequestedShipDate BETWEEN '<start>' AND '<end>'
GROUP BY SKU, SKUDesc, CustomerName
ORDER BY OpenBalance DESC;
```

8. Data Layer Recommendation

Use Case	Recommended Source
Order status, ship dates, qty, customer	vw_Orders_Base
Ordered vs. shipped reconciliation (OTIF)	vw_Orders_Base
Downstream joins to inventory / planning	vw_Orders_Base (via OrdersBaseKey)
Add hold codes to order query	vw_Orders_Base + lineholds CTE join
NP hold exception queue	vw_OpenOrdersWithNPHold
All active holds — display / triage	vw_OpenOrdersWithHold
City / State geography on orders	vw_Orders (display only — no aggregation)
Pricing strategy visibility	vw_Orders (display only — no aggregation)
Bailment / Control State orders	SYS21_OpenOrders (separate scope)

9. Known Risks & Open Items

- **Charge lines in product queries.** PALLETSLIPCHG and similar charge-type SKUs appear alongside product lines. These rows often return NULL for Company, Plant, Stockroom, and OrganizationCode. Always filter them out for inventory or quantity analysis.
- **vw_Orders duplicate rows.** The raw fulfill line join in vw_Orders produces duplicate rows when multiple fulfill lines exist per order line. Never use for quantity aggregation or order counts without deduplication.
- **Redundant columns.** CustomerNumber = AccountNumber and CustomerName = AccountName in all observed data. Use the Customer variants; treat Account variants as legacy.
- **Underscore-prefixed columns in vw_Orders.** _OrderQty and _LineUnitSellingPrice appear flagged or in transition. Confirm with jsimpson@sazerac.com before use.
- **Hold code join key for vw_Orders_Base.** The lineholds CTE joins on LINEID, which is not exposed in vw_Orders_Base. A direct join requires querying the raw line extract table alongside the view, or modifying the view to expose LineId. Validate before building hold-enriched queries.
- **Stockroom and LineSubInventory are NULL in observed data.** These fields likely populate only under specific allocation configurations or post-shipment. Do not rely on them as filter criteria without first confirming population logic.
- **All columns nullable except OrdersBaseKey.** Queries filtering on status, dates, or customer fields should handle NULLs explicitly.
- **vw_Orders is a parallel build, not derived from vw_Orders_Base.** Changes to vw_Orders_Base are not automatically reflected in vw_Orders or the hold views. These views require independent maintenance.
- **SYS21_OpenOrders not documented here.** A separate reference should be created for Bailment / Control State data.

Questions about this documentation or underlying views: jsimpson@sazerac.com